# 11 EFFECTIVE MODELLING

## OBJECTIVES

- Introduce Unified Modelling Language (UML).
- Discuss how various things can be modelled, specifically:
  - entities like classes, packages, nodes and components;
  - relationships like dependencies and associations;
  - processes like state changes and workflows;
  - user interaction.
- Give general advice on creating and using models.
- Explain how software professionals use modelling.

## RECAP

Modelling was first introduced in Chapter 4.

As Chapter 4 explained, abstractions – while useful for representing parts of a solution – can be difficult to get to grips with. Models offer a more concrete and manageable means of working with abstractions.

This chapter shows the various types of models you can create using Unified Modelling Language (UML) as an example.

A model represents the entities in a solution (or, more often, in part of a solution) and relationships between them. They typically include only details relevant in a certain context.

Models come in two types:

- Static models depict a system at a particular point in time.
- Dynamic models show how a system changes over time.

Use of models brings several advantages:

- They reduce the mental effort required to comprehend a solution.
- Some models are formal models and help to validate ideas by seeing if any rules are broken.
- Some models are executable models and predict how a solution will behave.

Different disciplines find particular types of model more helpful than others. Engineers focus on physical models that depict working systems. Scientists often favour mathematical models that reduce phenomena down to variables.

Software developers tend to use conceptual models that describe ideas, and they have many different modelling languages to choose from. Conceptual models are often specialised towards a particular sub-domain of computing, like modelling business processes, data structures or network layouts. While they can be very powerful in their place, this limits each language's applicability.

Conversely, a **general-purpose modelling language** can be used to describe any part of a software-intensive system. Today's pre-eminent general-purpose modelling language is UML, an industry standard for describing software systems. Since this chapter discusses modelling in general, it uses UML as an example of how to approach modelling in software solutions.

UML provides a dozen or so different types of diagrams. A complete description of them all lies outside the scope of this book. Instead, more in keeping with this book's focus, this chapter describes each major computational aspect of a solution and how it can be modelled. Specifically, it shows how UML can be used to model:

- entities;
- relationships;
- processes;
- usage.

Each aspect is illustrated with examples, along with corresponding source code listings that implement each example model.

If you wish to buy a book on UML, make sure it covers UML 2, the latest version. There is a wealth of books on UML, including:

*UML Distilled* (Fowler, 2004) and *UML 2 For Dummies* (Chonoles and Schardt, 2003).

## ENTITIES

Static models were introduced in the Chapter 4, section 'Static vs dynamic models'.

Entities are the focus of static diagrams (aka structural diagrams in UML), which depict components in a solution.

### Classes

A **class diagram** shows a solution decomposed into classes. It's one of the most important types of diagrams when developing an object-oriented solution.

The building block of a class diagram is a box representing the class itself. This can be as simple as shown in Figure 11.1.

**Figure 11.1 Simple class diagram**

While this is a valid diagram, it's not awfully helpful. After all, it corresponds to the following code:

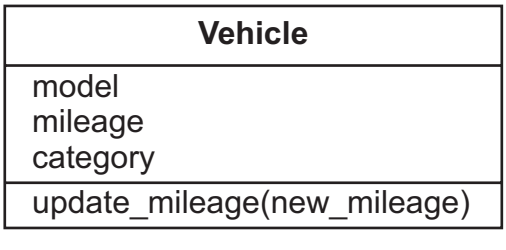
```
class Vehicle:
    pass
```

A depiction of a class showing only the name is perfectly fine, but we can add more detail to make it more informative. A class in a class diagram can have three sections:

- The **top** section contains the class's name.
- The **middle** section lists the attributes of the class.
- The **bottom** section lists the operations of the class.

Inspired by the vehicle rental solution from the previous chapter, let's fill in some of those details (see Figure 11.2).

**Figure 11.2 Class diagram with attributes and operations**



The diagram now tells us more about the class: we know it has several attributes and one operation called `update_mileage` (which takes one parameter called `new_mileage`). Again, the model can be transformed directly into source code:

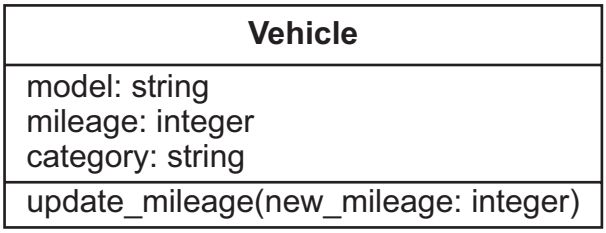
```
class Vehicle:
    def __init__(self):
        self.model = None
        self.mileage = None
        self.category = None

    def update_mileage(self, new_mileage):
        pass
```

Clearly, the model doesn't correspond to a fully working solution, but it's a valid piece of code that can serve as the starting point for complete implementation.

You can also include type information in a class diagram, like Figure 11.3.

**Figure 11.3 Class diagram with types included**




Although Python is **dynamically typed** (see the definition box below), it's still advisable to include type information in models for the sake of clarity.

**Dynamic typing**: a dynamically typed programming language doesn't require the programmer to specify the type of an object. Instead, the computer determines the object's type when it is assigned a value. For example, seeing `name = 'Keith'` tells the computer that `name` is a string. This stands in contrast to statically typed languages, where objects **must** have their types declared in the source code.

## Packages

Recall that Python programs can be organised into modules and packages. UML provides a way to model these too. In UML, a package looks like Figure 11.4.

It holds a collection of related objects, serving as a convenient way to organise your solution into sections. A package diagram depicts its contents like Figure 11.5.

**Figure 11.4 An empty package**

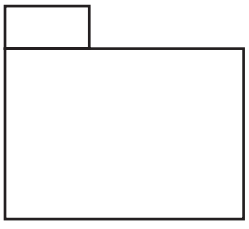
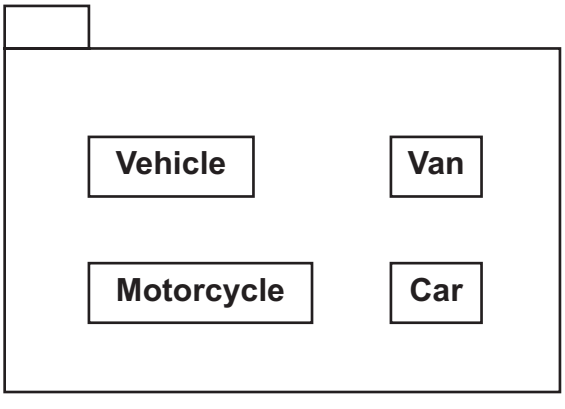


**Figure 11.5 A non-empty package**




Thus, it can represent a Python module. It can also represent a Python package if the corresponding UML package contains other packages.

## Components

A component is a complete, self-contained and replaceable piece of software that performs a function. An example of a component is an executable (like a .EXE file on Windows).

UML depicts a component like that seen in Figure 11.6.

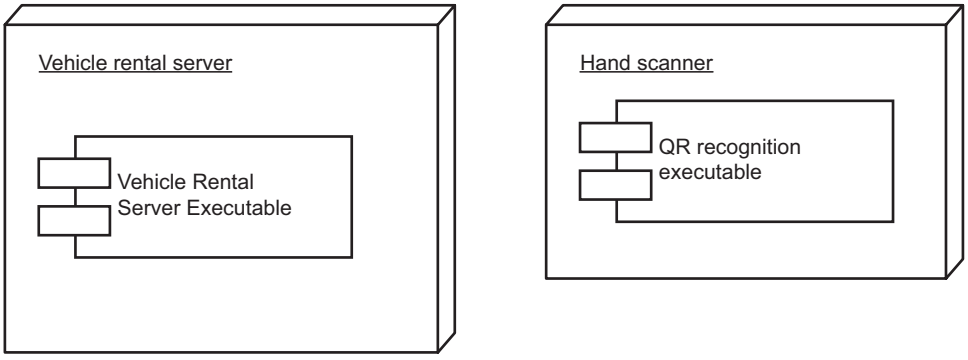
**Figure 11.6 A component**

### Nodes

When physical location is relevant in a solution, you might want to make that clear in a model. UML provides a way to do this via deployment diagrams. These model parts of a solution as **nodes**, where a node corresponds to a physical location.

As an example, let's say the vehicle rental system identifies vehicles in its fleet by a means of bar code. The user takes a handheld scanner and uses it to scan the bar code that is attached somewhere inside the vehicle. The scanner then relays this information back to the main system housed on a server somewhere. Thus, the overall system is necessarily divided between two locations. Figure 11.7 shows this.

**Figure 11.7 Examples of nodes**




## RELATIONSHIPS

Entities in a solution share various connections. UML provides ways of modelling relationships that attempt to capture all different possibilities. The most common relationships can be organised into two categories:

1. dependencies;
2. associations.

The various relationships in each category can be sorted in order of how much information they reveal.

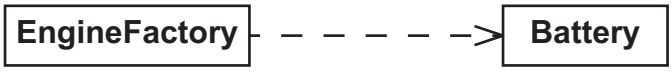
### Dependencies

Let's take **dependencies** first. Saying that entity B depends on entity A reveals little. It merely states that B requires A to provide some functionality in some way. It doesn't specify exactly how, but a dependency does imply that B is sensitive to changes in A.

As an example, look back at the `EngineFactory` in the previous chapter ('Design patterns' section). The `EngineFactory` has responsibility for creating objects found in a car engine, therefore it must know some things about how to create them. The `EngineFactory` therefore requires, for instance, a `Battery` type to exist and provide an initialiser. In other words, the `EngineFactory` depends on the `Battery` type and is sensitive to alterations.[64]

A dependency is depicted using a dashed arrow with an open arrowhead (see Figure 11.8).

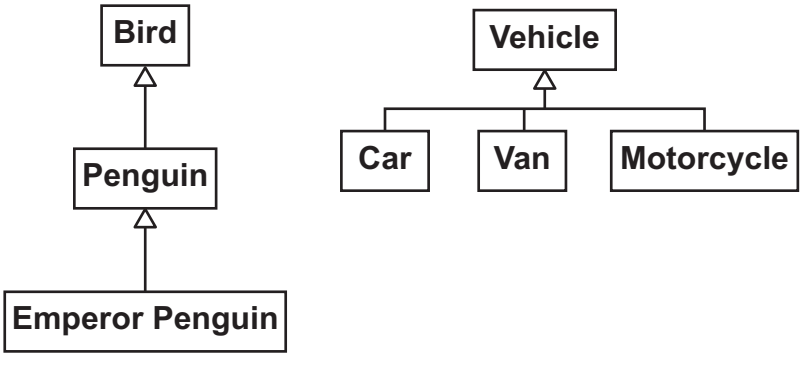
**Figure 11.8 EngineFactory depends on Battery**




Next, let's look at a more specific type of dependency: inheritance (first discussed in Chapter 10). Inheritance creates a specific sort of dependency between a type and its subtype, so that they share an 'is a' relationship. In our vehicle rental example, we said that a Ford Transit **is a** van, and that a van **is a** vehicle. In each case, the former is a specialisation of the latter[65] and inherits the attributes and capabilities of its parent (possibly overriding some of them in the process). Therefore, it says not only that the Van type depends on the Vehicle type, but it also specifies the nature of that dependency.

UML allows you to model this relationship. A unidirectional line with a solid white arrowhead depicts an inheritance (the parent class is the one being pointed at).

**Figure 11.9 Examples of inheritance**



The examples in Figure 11.9 correspond to the following code:

```
class Bird:
    pass

class Penguin(Bird):
    pass

class EmperorPenguin(Penguin):
    pass

class Vehicle:
    pass

class Car(Vehicle):
    pass

class Van(Vehicle):
    pass

class Motorcycle(Vehicle):
    pass
```

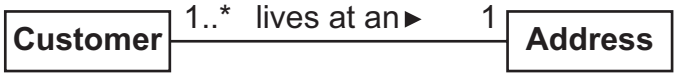
## Associations

Associations describe a relationship where either:

- entities share a link; or
- one entity aggregates other entities.

You can choose from a range of UML associations, depending on how much information you want the relationship to imply.

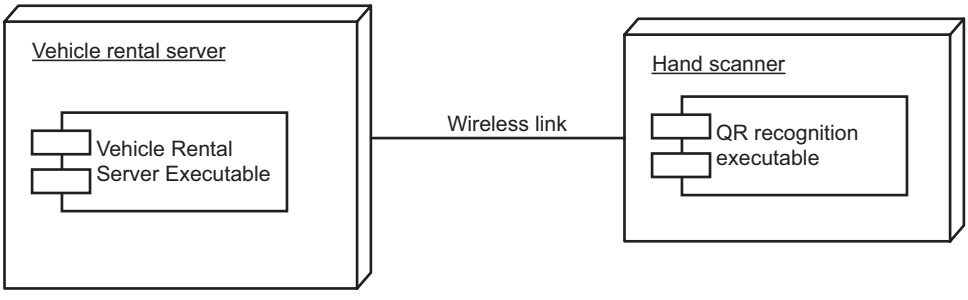
**Figure 11.10 An association between customers and addresses**




A simple association would look like something the arrangement in Figure 11.10. This models the relationship between customers and their addresses. It states that for each address the system contains, one or more customers are registered as living there, but says nothing more about the relationship.

Another example would be the link between two nodes. We might want to extend Figure 11.7 and include the communication between the hand scanner and central server. The model in Figure 11.11 makes this link explicit. Although the line might look like a cable between the two, it implies any sort of communication (hence we annotate the link to clarify what kind of interface is intended).

**Figure 11.11 A link between two nodes in a deployment diagram**

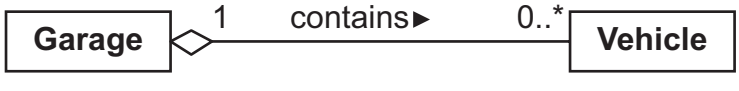



A simple association between classes reveals little about how that link might be implemented in source code. That's fine if you wish to leave that particular question open. However, should you wish to model associations in more detail, you can choose from more detailed versions.

An **aggregation** is an association that implies a loose relationship between two types. One may make use of the other, but their existences are independent of each other. For example, a garage may contain vehicles, but if you were to get rid of the garage, the vehicles would remain. The reverse is also true.

A UML aggregation is depicted like an association, except that the end of the line linking to the aggregating type has a clear diamond shape.

**Figure 11.12 An aggregation between a garage and vehicles**




The model in Figure 11.12 implies a relationship like this:

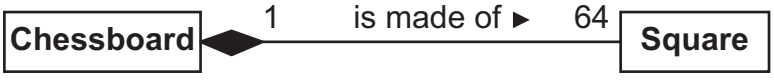
```
class Garage:
    def __init__(self):
        self.cars = []

class Car:
    pass

garage = Garage()
garage.cars.append(Car())
```

A **composition** is another sort of association, but a stronger one than an aggregation. It implies that an instance of one type is composed of instances of another and that the existence of the latter depends on the existence of the former. Getting rid of a composite object implicitly means getting rid of all its constituent objects. This means that the composite object has responsibility for creating its constituents (see Figure 11.13).

**Figure 11.13 A chessboard is composed of squares**




One way we could choose to implement this model in Python is like so:

```
class Square:
    def __init__(self, column, row):
        self.column = column
        self.row = row

    # The __repr__ function allows you to specify what an
    # object looks like when you print it.
    def __repr__(self):
        return '{}-{}'.format(self.column, self.row)

class Chessboard:
    def __init__(self):
        self.columns = ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H']

        # A nested list comprehension. This goes through
# every item in self.columns for each number 1 to 8. In
# other words, it goes through all 8 items of self.columns
# 8 times. Thus, it creates 64 Squares in a list.
        self.board = [Square(c, r) for r in range(1,9) for c in self. \
columns]

# When using a Chessboard, there's no need for you to create
# Squares because they're created automatically by the
# Chessboard.
c = Chessboard()
print(c.board)
```

The format method provided by a string allows you to put the current values of variables into that string. It goes through a string from left to right and adds the values of the corresponding parameters from left to right. In the preceding example, if a square was at column 3 and row 4, the __repr__ method would return "3-4".

A couple of things to note at this point:

- The code sample represents just one way to implement the model. A model rarely corresponds with only a single implementation, so once a model exists you still have implementation decisions to make.
- The difference between aggregation and composition varies in importance between programming languages. In a language where the creation and destruction of objects are managed by the computer (as it is in Python), it's not such an important distinction.

## PROCESSES

The modelling techniques seen so far deal with the static configuration of a system. That is, they don't take into account how things change over time. During execution of a program, things are constantly changing in complex ways, so it's important to be able to model that as well. This section shows how you can model dynamic behaviour in UML.

### State changes

Over time, certain elements of a system go through state changes: counters increase or get reset, switches open and close, messages are made to appear and then disappear. By carefully modelling these changes, you can predict how states change and under what circumstances, allowing you to verify that a system cannot enter an unexpected state (which is a key source of errors).

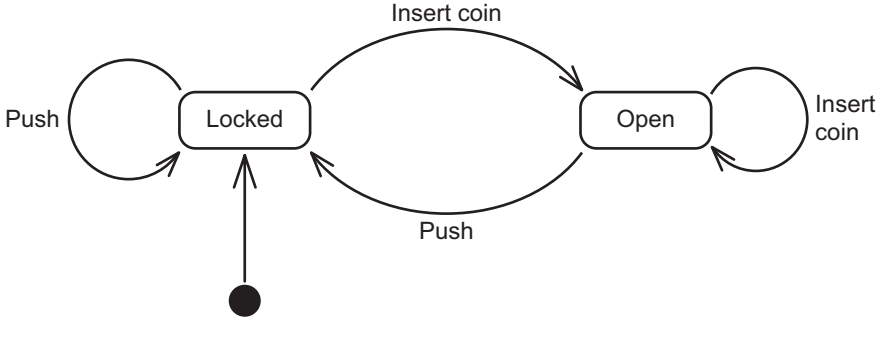
State machine diagrams were first mentioned in Chapter 4. See the section 'Static vs dynamic models'.

We've already seen an example of the standard method for modelling state changes in Chapter 4: the state machine diagram. Let's revisit the original diagram and examine exactly how it works.

The diagram in Figure 11.14 models the changes in state of a turnstile. It demonstrates the following elements of a state machine:

- **Initial state**: the black circle represents the starting point of the system. When the system is first activated, the transition from this point is executed.
- **Transitions**: an arrow depicts the movement of the system from one state to another. It can be labelled with the action that causes the transition (for example, inserting a coin causes the turnstile to become open, pushing a locked turnstile causes no change in state).
- **States**: a box with rounded edges depicts a state.

**Figure 11.14 State machine diagram of a coin-operated turnstile**



This shows a minimal amount of detail, but state machine diagrams offer additional elements.

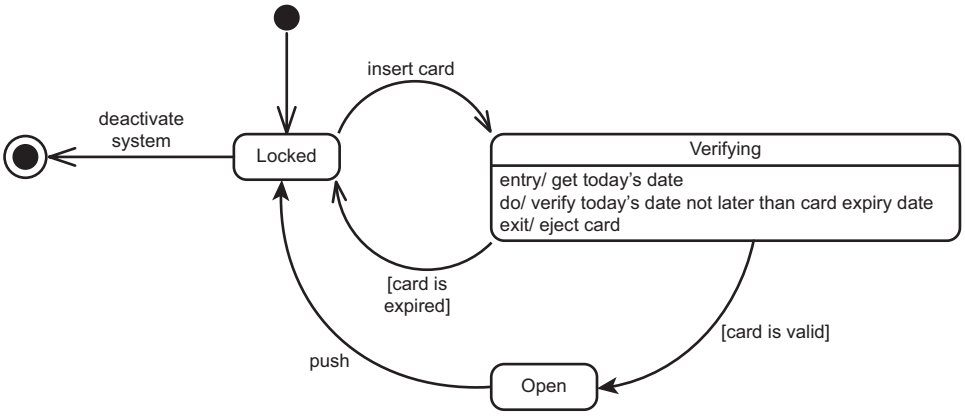
One thing that can be added is an **end state**, which represents a point from which no further changes in state are possible (usually meaning the system has shut down). An end state is depicted as a solid black circle surrounded by a larger empty circle.

But the main place where detail can be added is inside the states. A state box may display additional activities relating to that state in an optional bottom section. Each activity is prefixed by a term explaining what kind of activity it is:

- 'entry/' happens when the system enters this state;
- 'exit/' happens when the system leaves this state;
- 'do/': happens while the system is in this state.

To demonstrate these, let's change our turnstile from coin-operated to card-operated. See Figure 11.15.

**Figure 11.15 State machine diagram of a card-operated turnstile**




The access card stores an expiry date. When it's inserted into the turnstile, the system compares today's date with the expiry date. If the card has not yet expired, then it opens the turnstile, otherwise the turnstile remains locked. In either case, the card is ejected and given back to the user.

The transitions leaving the 'Verifying' state are expressed as **guard conditions** – visible by being encased in square brackets. Rather than being an event that triggers the state change, a guard condition is a truth statement that has to be true for that transition to be triggered.

Finally, when the system ceases to be used, it can be shut down by owner, putting it into the end state.

### Workflows

A state machine diagram emphasises the changes in state of a single entity, essentially showing progress from one object's point of view. However, you sometimes might want to model the flow of control through your solution, emphasising the various decisions and activities, and seeing things from a more global perspective. A model that does this is called a workflow, and can be constructed using a UML activity diagram.

These are the individual parts of an activity diagram:

- A black circle and an encircled black circle represent the **start** and **end states** respectively.
- Rounded rectangles represent **actions**.
- Diamond shapes represent **decisions** (i.e. conditionals).
- Thick bars represent the start and end of **concurrent activities**.
- Arrows represent the **flow of control**. Similar to those in state machine diagrams, arrows in activity diagrams can optionally have labels (plain text) or guard conditions (truth statements in square brackets) representing necessary conditions for following that particular flow.

Figure 11.16 shows an activity diagram[66] that depicts the same card-operated turnstile from Figure 11.15.

When choosing between a state machine and an activity diagram, keep the following in mind:

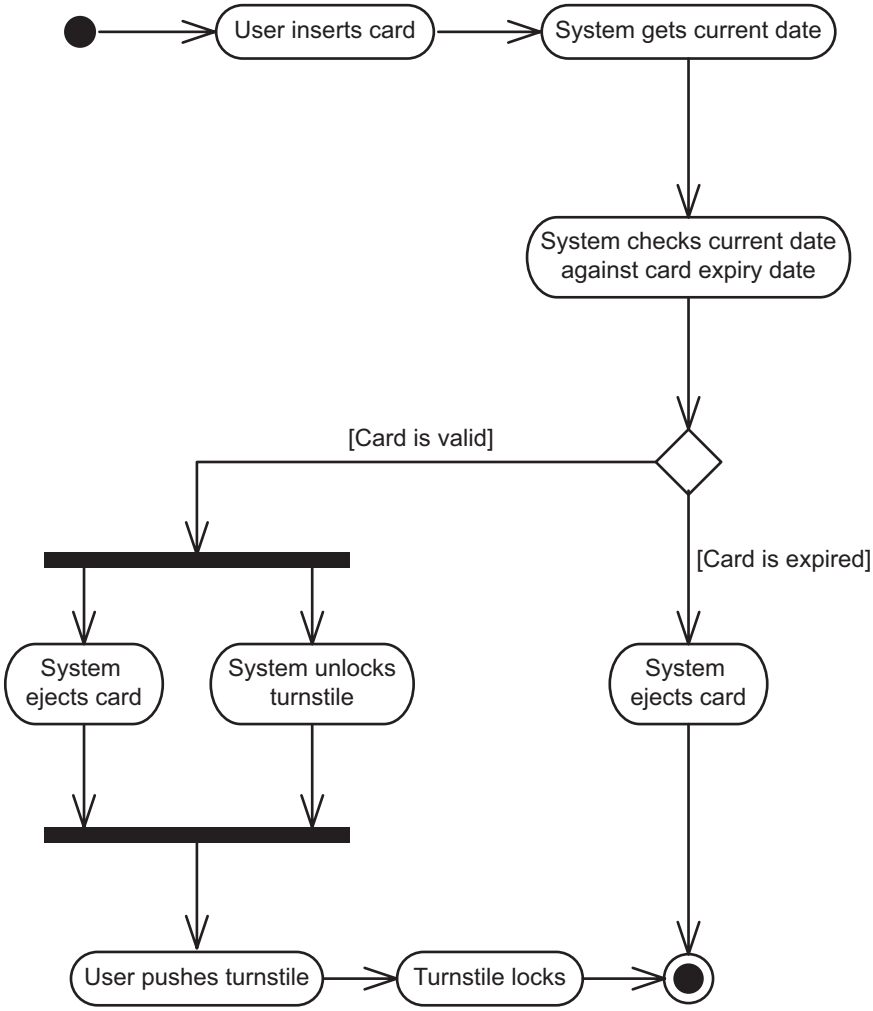
- A state machine diagram gives the view of a system from an object's point of view – other participating objects are de-emphasised. This makes it particularly useful when your goal is to describe an object's behaviour.
- An activity diagram gives a much more holistic view of a process. The cost is that it tends to describe each object's behaviour only partially. This makes it useful when your goal is to describe a procedure.

## USAGE

The models seen so far depict a system in terms of its constituent parts, modelling it from the developer's point of view. However, an important perspective is missing from those examples: the user's.

When looking at a system from the user's perspective, we switch from asking **how does the system work?** to instead asking **what can someone do with the system?**

**Figure 11.16 Activity diagram of a card-operated turnstile**

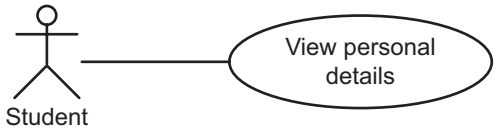



## Use case diagrams

**Use case**: an action (or sequence of actions) provided by a system that helps a user achieve a goal.

A use case diagram represents a user's interaction with a system. It connects different types of user to a set of available use cases (see the box above). The goal of the diagram is to show who can do what with a system. It does **not** show how those use cases actually work – that's the job of all the other types of models seen so far. Consequently, use case diagrams are particularly useful in the early phases of constructing a solution because they help to clarify system requirements.

**Figure 11.17 Simple use case diagram**




A minimal use case diagram might look something like Figure 11.17. It has the following parts:

- The stick figure (properly called an **actor**) represents not just any user, but a particular type of user. The type itself is named beneath the actor.
- The ellipse represents a **use case**.
- The line is an **association** and connects the actor to one or more use cases, indicating that the actor is involved in the corresponding use case.

Figure 11.17 describes a use case from a student database system. It shows that users who are considered as students are able to view their personal details. Not extraordinarily complex, you might say. Let's add some more to it.

**Figure 11.18 More complex use case diagram**

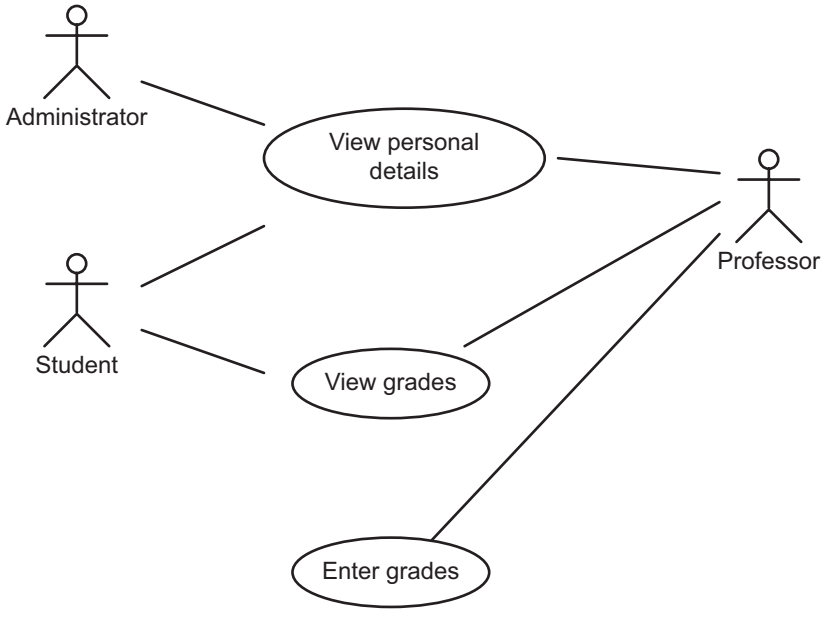


Figure 11.18 shows more of the system's features. Here are the most significant points:

- The system recognises three types of user: student, administrator and professor. The system restricts a user to performing only a certain set of actions depending on what type they are.
- Students can view their own personal details and grades. For obvious reasons, they can't enter grades into the system.
- Administrators can look up students' personal details, but grades are a private matter between student and professor, so they can neither view nor edit them.
- Professors have most power in the system. They can carry out all listed use cases.

## GENERAL ADVICE

Some tips about modelling in general were initially discussed in Chapter 4, section 'Things to remember'.

### Characteristics of an effective model

An effective model has the following characteristics (Selic, 2003):

- **Abstract**: a good model hides unimportant and irrelevant details, allowing us to concentrate on the important concepts.
- **Understandable**: a good model presents information intuitively and requires less effort to understand than the equivalent code does.
- **Accurate**: a good model must be a true-to-life representation of the system it models.
- **Predictive**: a good model allows us to correctly predict the non-obvious properties of a system.
- **Inexpensive**: it should be cheaper to produce a model than to construct the actual system itself.

### The purpose of modelling

Keep in mind that, ultimately, making models is about conveying detailed ideas in a simplified form. A model is only useful if it communicates useful information to an audience, or makes the job of creating a solution easier.

Always ask yourself **why** you're making a model. Guard against creating a model simply for its own sake. If a model isn't helping you or it has no wider audience, then you're probably wasting your time.

Choose the form of your model carefully. This decision is driven by the purpose and the audience. Ask:

- Which entities, relationships or processes are you trying to understand? Which type of model is best suited to those?
- Who or what is your audience, and which types of model are most suitable for them?

One of the key advantages of creating models is that we can express our solution in ways that are not bound to any particular implementation. The same UML model can be implemented unchanged in dozens of different languages using any number of different techniques and technologies.

## Modelling among the professionals

For all its benefits, to what extent do software professionals put modelling into practice?

While modelling does see plenty of usage in industry, it's not exactly ubiquitous. Several reasons might account for this, but perhaps the key reason is that there exists a tension between models and source code. A class in source code is itself a kind of model, one which represents a real-world object or concept. Modelling a class is, therefore, to model a model. Hence, it's tempting to view software modelling as extraneous effort. Furthermore, whereas a class **must** be updated when the program behaviour requires changing, there's no necessity to update the corresponding model, leaving it to become out of date.

Some experts advocate a closer link between models and code, for example in the form of model-driven development (Selic, 2003; Hailpern and Tarr, 2006). This would make models become the focus of a programmer's attention instead of source code (the source code being automatically generated from the model definition), but only time will tell whether industry practice goes on to follow this path.

Research into how modelling is done by industry today (for example, Selic, 2003; Petre, 2013) reveals certain patterns among the professionals:

- No particular type of model sees widespread usage. While an organisation may use various types of models, research shows that each model type sees frequent use by only a minority of organisations.
- Modelling often plays a **secondary role**, possibly because it's very easy to 'dive into' coding, as opposed to other disciplines where building something is a big investment and needs careful up-front planning.
- A study of UML usage suggests it's mostly used **informally** and **selectively**, including as:
  - a 'thought tool' for thinking about the concepts early on, but not carrying through to design;
  - a way to communicate with technical stakeholders;
  - a communication device when collaborating with others, especially because it helps overcome human-centric difficulties (such as spoken language and culture).

- Models are often used in a focused way, representing only small parts of the system in isolation. Creating large, overly comprehensive diagrams is avoided.
- Many professionals don't particularly value **exact notation** or **excessive detail**.
  - Models with a lot of detail risk overwhelming the audience. Experienced practitioners prefer to keep models **simple**.
  - In some cases, professionals **adapt** UML to the task at hand and don't fully respect the rules and notation.

## SUMMARY

Aspects of a software solution that are typically modelled include entities, relationships, processes and usage. Models can sometimes be used to generate equivalent source code, albeit code that is partial and requires the programmer to add what's missing.

Various types of modelling languages exist. Some are very specific and intended to show only limited aspects of a system. Others (like UML) are more generic and can be used to describe almost any aspect of a system.

The effort required to produce a model should be justifiable. In practice, for a model to be useful, it should be abstract, understandable, accurate, predictive and inexpensive to produce.

## EXERCISES

### EXERCISE 1

Mark the following statements as true or false:

A. Static models show how entities change over time.
B. The middle section of a class in a class diagram lists attributes.
C. Inheritance is modelled using an arrow with a solid, white arrowhead.
D. Aggregation and composition are types of dependency.
E. A state machine diagram emphasises the changes in state of a single object.

### EXERCISE 2

Look at Figure 11.13. Extend the diagram to include chess pieces.

**Hint:** you should add another class called ChessSet to tie everything together.

## EXERCISE 3

Draw a use case diagram for a typical, high street cash machine. Differentiate between ordinary users and customers of that bank's machine (who can thus access additional features like printing bank statements or paying in money).

## EXERCISE 4

Draw an activity diagram for the process of withdrawing money from a cash machine.

## EXERCISE 5

There have been complaints about the card-operated turnstile gate (see section on 'Workflows'). Some people forget their card and leave it in the slot. Alter the activity diagram so that this can no longer happen.